

for each user, calculate the cumulative sum of activity counts for each activity type over time, ordered by activity date

columns - user_id, activity_date, activity_type, activity_count

```
%python
from pyspark.sql.window import Window
import pyspark.sql.functions as F

window_spec = Window.partitionBy("user_id",
"activity_type").orderBy("activity_date").rowBetween(Window.UnboundedPreceding,
Window.CurrentRow)
df = df.withColumn("cumulative_sum",
F.sum("activity_count").over(window_spec)).orderBy("user_id", "activity_type",
"activity_date")
```

```
%sql
select user_id, activity_type, activity_count, sum(activity_count) over (partition by
user_id, activity_type order by activity_date rows between unbounding preceding
and current row) as cumulative_sum from user_activites order by user_id,
activity_type, activity_date;
-----
```

identify the top_selling product, characterized by the highest revenue, within each category for the last quarter?

orders - order_id, customer_id, order_date, total_amount
products - product_id, product_name, category
order_detail - order_detail_id, order_id, product_id, quantity, unit_price

```
%python
from pyspark.sql.functions import col, sum, max, when

from datetime import datetime, timedelta
current_date = datetime.now()
last_quarter_start = current_date - timedelta(days = 90)

result = orders.join(order_detail, "order_id").join(products,
"product_id").filter(col("order_date") >= last_quarter_start)

revenue_by_product = result.groupBy("category", "product_id",
"product_name").agg(sum(col("quantity") * col("unit_price")).alias("revenue"))

window_spec = Window.partitionBy("category").orderBy(col("revenue").desc())

ranked_products = revenue_by_product.withColumn("rn",
F.row_number().over(window_spec)).filter(col("rn") == 1).drop("rn")

%sql
with last_quarter_date as (
select p.category, p.product_id, p.product_name, sum(od.quantity * od.unit_price) as
revenue from orders o
join order_detail od on o.order_id = od.order_id
join products p on od.product_id = p.product_id where o.order_date >=
add_months(trunc(sysdate, 'Q'), -3) group by p.category, p.product_id,
```

```

p.product_name)
ranked_products as (
select *, row_number() over (partition by category order by revenue desc) as rn from
last_quarter_data)
select category,product_id, product_name, revenue from ranked_products where rn
= 1;

```

write a sql query to fetch top5 selling products for each day

```

orders - order_id, customer_id, order_date, total_amount
orders_item - order_item_id, order_id, product_id, quantity, unit_price
products - product_id, product_name, category_id, price

```

```

%python
from pyspark.sql.window import Window
daily_sales = orders.join(orders_item, "order_id").join(products,
"product_id").groupBy(F.date_format("order_date", "yyyy-MM-dd").alias("sales_date"),
"product_id", "product_name").agg(F.sum("quantity").alias("total_quantity_sold"))

window_spec =
Window.partitionBy("sale_date").orderBy(col("total_quantity_sold").desc())

ranked_products = daily_sales.withColumn("rank",
F.row_number().over(window_spec)).filter(col("rank") <= 5).orderBy("sale_date", "rank")

```

```

%sql
with daily_product_sales as (
select trunc(o.order_date) as sale_date, p.product_id, p.product_name,
sum(oi.quantity) as total_quantity_sold from orders o
join orders_item oi on o.order_id = oi.order_id
join products p on oi.product_id = p.product_id group by trunc(o.order_date),
p.product_id, p.product_name),
ranked_daily_products as (
select *, row_number() over (partition by sale_date order by total_quantity_sold desc)
as rank from daily_product_sales)
select sale_date, product_id, product_name, total_quantity_sold, rank from
ranked_daily_products where rank <= 5 order by sale_date, rank;

```

write a SQL query to fetch rides with incomplete transactions on a uber platform

```

rides - ride_id, driver_id, user_id, start_time, end_time, start_location, end_location,
fare_amount, ride_distance
payment_transactions - transaction_id, ride_id, transaction_time, transaction_amount

```

```

%python
# Method 1: Rides without any transaction
incomplete_rides_1 = rides.join(
    payment_transactions.select("ride_id").distinct(),
    rides.ride_id == payment_transactions.ride_id,
    "left_anti"
)

```

```
# Method 2: Rides with transaction amount mismatch
incomplete_rides_2 = rides.join(payment_transactions, "ride_id", "left") \
    .groupBy(
        rides.ride_id,
        rides.driver_id,
        rides.user_id,
        rides.fare_amount
    ) \
    .agg(
        F.sum("transaction_amount").alias("total_paid"),
        F.count("transaction_id").alias("transaction_count")
    ) \
    .filter(
        (col("total_paid").isNull()) |
        (col("total_paid") < col("fare_amount")) |
        (col("transaction_count") == 0)
    )
```

```
# Method 3: Detailed check
incomplete_rides_3 = rides.join(
    payment_transactions.groupBy("ride_id")
        .agg(
            F.sum("transaction_amount").alias("total_paid"),
            F.count("*").alias("txn_count")
        ), "ride_id", "left") \
    .filter(
        (col("total_paid").isNull()) | # No transaction
        (col("total_paid") < col("fare_amount")) | # Partial payment
        (col("txn_count") == 0) # No transaction records
    )
```

```
%sql
-- Complete solution with all details
WITH ride_payments AS (
    SELECT
        r.ride_id,
        r.driver_id,
        r.user_id,
        r.start_time,
        r.end_time,
        r.fare_amount,
        SUM(pt.transaction_amount) as total_paid,
        COUNT(pt.transaction_id) as transaction_count,
        CASE
            WHEN COUNT(pt.transaction_id) = 0 THEN 'NO_TRANSACTION'
            WHEN SUM(pt.transaction_amount) < r.fare_amount THEN 'PARTIAL_PAYMENT'
            WHEN SUM(pt.transaction_amount) > r.fare_amount THEN 'OVER_PAYMENT'
            ELSE 'COMPLETE'
        END as payment_status
    FROM rides r
    LEFT JOIN payment_transactions pt ON r.ride_id = pt.ride_id
    GROUP BY r.ride_id, r.driver_id, r.user_id, r.start_time,
             r.end_time, r.fare_amount
)
SELECT *
FROM ride_payments
```

```
WHERE payment_status IN ('NO_TRANSACTION', 'PARTIAL_PAYMENT')
ORDER BY start_time DESC;
```

Astronauts - astronaut_id, name, nationality, birth_date
mission - mission_id, mission_name, start_date, end_date
mission_crew_table - mission_id, astronaut_id, role
space_stations - station_id, station_name, launch_date, deorbit_date
station_visit - visit_id, mission_id, station_id, docking_date, undocking_date

1. List all astronauts who have participated in more than one mission

```
%python
from pyspark.sql import functions as F

# Count missions per astronaut
missions_per_astronaut = mission_crew_table.groupBy("astronaut_id") \
    .agg(F.countDistinct("mission_id").alias("mission_count")) \
    .filter(F.col("mission_count") > 1)

# Join with astronaut details
result = missions_per_astronaut.join(
    astronauts_df,
    missions_per_astronaut.astronaut_id == astronauts_df.astronaut_id
).select(
    astronauts_df.astronaut_id,
    "name",
    "nationality",
    "birth_date",
    "mission_count"
).orderBy(F.col("mission_count").desc(), "name")

print(f'Astronauts with multiple missions: {result.count()}')
result.show()
```

```
%sql
-- Astronauts who participated in more than one mission
SELECT
    a.astronaut_id,
    a.name,
    a.nationality,
    a.birth_date,
    COUNT(DISTINCT mct.mission_id) as mission_count
FROM astronauts a
JOIN mission_crew_table mct ON a.astronaut_id = mct.astronaut_id
GROUP BY a.astronaut_id, a.name, a.nationality, a.birth_date
HAVING COUNT(DISTINCT mct.mission_id) > 1
ORDER BY mission_count DESC, a.name;
```

2. Find the top 3 countries with the most cumulative days spent in space by their astronauts

```
%python
from pyspark.sql import functions as F
```

```

# Calculate mission duration in days
mission_durations = mission_df.select(
    "mission_id",
    "start_date",
    "end_date"
).withColumn(
    "mission_duration_days",
    F.datediff(F.col("end_date"), F.col("start_date"))
)

# Join with crew and astronauts to get nationality
space_time_by_astronaut = mission_crew_table.join(
    mission_durations, "mission_id"
).join(
    astronauts_df, "astronaut_id"
).groupBy("astronaut_id", "nationality") \
    .agg(F.sum("mission_duration_days").alias("total_days_in_space"))

# Aggregate by country
space_time_by_country = space_time_by_astronaut.groupBy("nationality") \
    .agg(
        F.sum("total_days_in_space").alias("total_days"),
        F.countDistinct("astronaut_id").alias("astronaut_count"),
        F.avg("total_days_in_space").alias("avg_days_per_astronaut")
    ).orderBy(F.col("total_days").desc()) \
    .limit(3)

print("Top 3 countries by cumulative days in space:")
space_time_by_country.show()

```

```

%sql
-- Top 3 countries with most cumulative days in space
WITH mission_durations AS (
    SELECT
        mission_id,
        end_date - start_date as mission_duration_days
    FROM mission
    WHERE end_date IS NOT NULL
),
astronaut_space_time AS (
    SELECT
        a.nationality,
        a.astronaut_id,
        a.name,
        SUM(md.mission_duration_days) as total_days
    FROM mission_crew_table mct
    JOIN mission_durations md ON mct.mission_id = md.mission_id
    JOIN astronauts a ON mct.astronaut_id = a.astronaut_id
    GROUP BY a.nationality, a.astronaut_id, a.name
),
country_stats AS (
    SELECT
        nationality,
        SUM(total_days) as total_days_in_space,
        COUNT(DISTINCT astronaut_id) as astronaut_count,

```

```

        ROUND(AVG(total_days), 2) as avg_days_per_astronaut,
        ROW_NUMBER() OVER (ORDER BY SUM(total_days) DESC) as rank
    FROM astronaut_space_time
    GROUP BY nationality
)
SELECT
    nationality,
    total_days_in_space,
    astronaut_count,
    avg_days_per_astronaut
FROM country_stats
WHERE rank <= 3
ORDER BY total_days_in_space DESC;

```

3. Find astronauts who have visited at least 3 different space stations, and rank them based on their total time spent in space across all missions.

```

%python
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Calculate mission duration
mission_durations = mission_df.select(
    "mission_id",
    F.datediff("end_date", "start_date").alias("mission_days")
)

# Get station visits per astronaut
station_visits = station_visit_df.join(
    mission_crew_table, "mission_id"
).select(
    "astronaut_id",
    "station_id"
).distinct()

# Count stations per astronaut
astronauts_multiple_stations = station_visits.groupBy("astronaut_id") \
    .agg(F.countDistinct("station_id").alias("stations_visited")) \
    .filter(F.col("stations_visited") >= 3)

# Calculate total space time
total_space_time = mission_crew_table.join(
    mission_durations, "mission_id"
).groupBy("astronaut_id") \
    .agg(F.sum("mission_days").alias("total_days_in_space"))

# Join all information
astronaut_ranking = astronauts_multiple_stations.join(
    total_space_time, "astronaut_id"
).join(
    astronauts_df, "astronaut_id"
).select(
    astronauts_df.astronaut_id,
    "name",
    "nationality",

```

```

    "stations_visited",
    "total_days_in_space"
).orderBy(F.col("total_days_in_space").desc())

# Add rank
window_spec = Window.orderBy(F.col("total_days_in_space").desc())
ranked_astronauts = astronaut_ranking.withColumn(
    "rank", F.row_number().over(window_spec)
)

print("Astronauts who visited 3+ space stations, ranked by total space time:")
ranked_astronauts.show()

```

```

%sql
-- Astronauts who visited 3+ space stations, ranked by total space time
WITH astronaut_station_visits AS (
    SELECT
        mct.astronaut_id,
        COUNT(DISTINCT sv.station_id) as stations_visited
    FROM station_visit sv
    JOIN mission_crew_table mct ON sv.mission_id = mct.mission_id
    GROUP BY mct.astronaut_id
    HAVING COUNT(DISTINCT sv.station_id) >= 3
),
astronaut_space_time AS (
    SELECT
        mct.astronaut_id,
        SUM(m.end_date - m.start_date) as total_days_in_space
    FROM mission_crew_table mct
    JOIN mission m ON mct.mission_id = m.mission_id
    WHERE m.end_date IS NOT NULL
    GROUP BY mct.astronaut_id
)
SELECT
    ROW_NUMBER() OVER (ORDER BY ast.total_days_in_space DESC) as rank,
    a.astronaut_id,
    a.name,
    a.nationality,
    asv.stations_visited,
    ast.total_days_in_space,
    ROUND(ast.total_days_in_space / 365, 2) as total_years_in_space
FROM astronaut_station_visits asv
JOIN astronaut_space_time ast ON asv.astronaut_id = ast.astronaut_id
JOIN astronauts a ON asv.astronaut_id = a.astronaut_id
ORDER BY ast.total_days_in_space DESC;

```

write a sql query to determine the percentage of buyers who bought airpods directly after they bought iphones.

transaction - transaction_id, customer_id, product_name, transaction_timestamp

```

%SQL
with customer_transactions as (
select customer_id, product_name, transaction_timestamp, lag(product_name) over

```

```
(partition by customer_id order by transaction_timestamp) as prev_product from
transactions where product_name in ('iPhone', 'AirPods')),
airpods_after_iphone as (
select distinct customer_id from customer_transactions where product_name =
'AirPods' and prev_product = 'iPhone'),
total_iphone_buyers as (
select count(distinct customer_id) as total_count from transactions where
product_name = 'iPhone')
select count(a.customer_id) as airpods_buyers_count, t.total_count as
iphone_buyers_count, round(count(a.customer_id) * 100.0/t.total_count,2) as
percentage from total_iphone_buyers t left join airpods_after_iphone a on t.customer_id = a.customer_id group
by t.total_count;
```

```
%python
result = transactions_df.alias("t1").join(transactions_df.alias("t2"),
(F.col("t1_customer_id") == F.col("t2_customer_id")) &
(F.col("t2_transaction_timestamp") > F.col("t1_transaction_timestamp"))) &
(F.col("t1_product_name") == "iPhone") & (F.col("t2_product_name") ==
"AirPods")).groupBy("t1_customer_id").agg(F.min("t2_transaction_timestamp").alias("airpods_purchase_time")).count()
```

```
total_iphone = transactions_df.filter(F.col("product_name") ==
"iPhone").select(customer_id).distinct().count()
percentage = (result/total_iphone) * 100 if total_iphone > 0 else 0 ;
----
```

```
season - sessionID, year
teams - teamID, TeamName
standings - sessionID, Position
```

1. write a SQL query to find the seasons where RCB ended as runners_up, showcasing their near misses at the championship.
print the season year, team name, team ID, team position

```
%python
rcb_team_id = teams_df.filter(F.col("TeamName") == "RCB").select("teamID").first()[0]
rcb_runner_up = standings_df.join(season_df,
"sessionID").join(teams_df.filter(F.col("teamID") == rcb_team_id),
"teamID").filter(F.col("Position") == 2).select("year", "TeamName",
"teamID", "Position").orderBy("year")
rcb_runner_up_seasons.show()
```

```
%sql
select s.year, t.TeamName, t.teamID, st.Position from season s join standings st on
s.sessionID = st.sessionID join teams t on st.teamID = t.teamID where t.TeamName = 'RCB'
and st.Position = 2;
```

2. find out which seasons saw RCB players winning the orange cap but the team failing to clinch the title. A player wins, Orange Cap when he scores the most number of runs in the whole tournament.

```
%python

rcb_team_id = teams_df.filter(F.col("TeamName") == "RCB").select("teamID").first()[0]

orange_cap_seasons = orange_cap_df.join(
```



```

players_df, "playerID").filter(
  F.col("teamID") == rcb_team_id).select("seasonID").distinct()

non_winning_seasons = standings_df.join(
  teams_df.filter(F.col("TeamName") == "RCB"), "teamID").filter(
  F.col("Position") > 1).select("sessionID").distinct()

result_seasons = orange_cap_seasons.join(
  non_winning_seasons,
  orange_cap_seasons.sessionID == non_winning_seasons.sessionID).join(
  season_df, orange_cap_seasons.sessionID == season_df.sessionID).select("year",
  F.lit("RCB").alias("TeamName"),
  F.lit(rcb_team_id).alias("teamID")).orderBy("year")

result_seasons.show()

%sql
WITH rcb_team AS (
  SELECT teamID FROM teams WHERE TeamName = 'RCB'),
rcb_orange_cap_seasons AS (
  SELECT DISTINCT oc.sessionID
  FROM orange_cap_winners oc
  JOIN players p ON oc.playerID = p.playerID
  WHERE p.teamID IN (SELECT teamID FROM rcb_team)),

rcb_non_winning_seasons AS (
  SELECT DISTINCT st.sessionID
  FROM standings st
  JOIN rcb_team r ON st.teamID = r.teamID
  WHERE st.Position > 1)
SELECT
  s.year,
  'RCB' as TeamName,
  r.teamID FROM season s
JOIN rcb_orange_cap_seasons oc ON s.sessionID = oc.sessionID
JOIN rcb_non_winning_seasons nw ON s.sessionID = nw.sessionID
JOIN rcb_team r ON 1=1 ORDER BY s.year;

```

write a SQL code to find the order_id of all imbalanced orders. An order is imbalanced if the maximum quantity of any product in the order is strictly greater than the average quantity of all products in that order.

OrderDetails - order_id, product_id, quantity

```

%python
from pyspark.sql import functions as F

# Calculate per-order statistics
order_stats = OrderDetails.groupBy("order_id").agg(
  F.max("quantity").alias("max_quantity"),
  F.avg("quantity").alias("avg_quantity"),
  F.count("*").alias("product_count"))

# Identify imbalanced orders

```

```

imbalanced_orders = order_stats.filter(
    F.col("max_quantity") > F.col("avg_quantity")).select("order_id")

# Alternative: Single query with window function
imbalanced_orders_alt = OrderDetails.groupBy("order_id").agg(
    F.max("quantity").alias("max_qty"),
    F.avg("quantity").alias("avg_qty")).filter(F.col("max_qty") >
F.col("avg_qty")).select("order_id")

# Show results with details
imbalanced_orders_with_details = OrderDetails.join(
    imbalanced_orders, "order_id").orderBy("order_id", "product_id")

imbalanced_orders_with_details.show()

%sql
SELECT
    order_id,
    MAX(quantity) as max_quantity,
    AVG(quantity) as avg_quantity,
    COUNT(*) as product_count
FROM OrderDetails GROUP BY order_id HAVING MAX(quantity) > AVG(quantity)
ORDER BY order_id;

```

find a solution to find managers with atleast five direct reports

```

%python
from pyspark.sql import functions as F

# Assuming employee table has: employee_id, manager_id, name
df = spark.read.table("employee")

# Count direct reports for each manager
manager_reports = df.groupBy("manager_id") \
    .agg(F.count("*").alias("direct_reports_count")) \
    .filter(F.col("direct_reports_count") >= 5) \
    .withColumnRenamed("manager_id", "employee_id")

# Join to get manager details
result = manager_reports.join(
    df.select("employee_id", "name"),
    "employee_id").select("employee_id", "name", "direct_reports_count") \
    .orderBy(F.col("direct_reports_count").desc())

result.show()

%sql
SELECT
    manager_id as employee_id,
    e.name as manager_name,
    COUNT(*) as direct_reports_count
FROM employee e1
JOIN employee e ON e1.manager_id = e.employee_id
WHERE e1.manager_id IS NOT NULL

```

```
GROUP BY manager_id, e.name
HAVING COUNT(*) >= 5
ORDER BY direct_reports_count DESC;
```

write an Pyspark and SQL query to report all the consecutive available seats in cinema.

cinema_table - seat_id, free

```
%python
```

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F
```

```
# Create windows for checking consecutive seats
```

```
window_spec = Window.orderBy("seat_id")
```

```
# Add flags for consecutive free seats
```

```
df = df.withColumn("prev_free",
                  F.lag("free").over(window_spec)) \
    .withColumn("next_free",
               F.lead("free").over(window_spec)) \
    .withColumn("seat_group",
               F.when(
                   (F.col("free") == 1) &
                   ((F.col("prev_free") == 1) | (F.col("next_free") == 1)),
                   1
               ).otherwise(0))
```

```
# Filter and group consecutive seats
```

```
consecutive_seats = df.filter(F.col("seat_group") == 1) \
    .select("seat_id", "free") \
    .orderBy("seat_id")
```

```
%sql
```

```
SELECT seat_id
FROM (
    SELECT
        seat_id,
        free,
        LAG(free) OVER (ORDER BY seat_id) as prev_free,
        LEAD(free) OVER (ORDER BY seat_id) as next_free
    FROM cinema_table) t WHERE free = 1 AND (prev_free = 1 OR next_free = 1) ORDER
BY seat_id;
```

write an SQL query to report the device that is first logged in for each player activity - player_id, device_id, event_date, games_played

```
%python
```

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F
```

```
window_spec = Window.partitionBy("player_id").orderBy("event_date")
```

```
# Get first login for each player
```

```
first_login = df.withColumn("rn", F.row_number().over(window_spec)) \
```

```
.filter(F.col("rn") == 1) \
.select("player_id", "device_id", "event_date") \
.withColumnRenamed("event_date", "first_login_date")
```

```
%sql
SELECT
    player_id,
    device_id,
    event_date as first_login_date
FROM (
    SELECT
        player_id,
        device_id,
        event_date,
        ROW_NUMBER() OVER (
            PARTITION BY player_id
            ORDER BY event_date) as rn FROM activity) t WHERE rn = 1 ORDER BY player_id;
```

write an SQL query to report the fractions of players that logged in again on the day after the day they first logged in, rounded to 2 decimal places. in other words , you need to count the number of players that logged in for at least two consecutive days starting from their first login date, then divide that number by the total number of players

activity - player_id, device_id, event_date, games_played

```
%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Get first login for each player
first_login = df.groupBy("player_id") \
    .agg(F.min("event_date").alias("first_login_date"))

# Get next day login
df_with_next_day = df.alias("a").join(
    df.alias("b"),
    (F.col("a.player_id") == F.col("b.player_id")) &
    (F.date_add(F.col("a.event_date"), 1) == F.col("b.event_date"))
).select("a.player_id", "a.event_date").distinct()

# Find players who logged in the day after first login
players_with_consecutive = first_login.join(
    df_with_next_day,
    (first_login.player_id == df_with_next_day.player_id) &
    (F.date_add(first_login.first_login_date, 1) == df_with_next_day.event_date)
).select("player_id").distinct()

# Calculate total players
total_players = df.select("player_id").distinct().count()

# Count players with consecutive login
players_with_consecutive_count = players_with_consecutive.count()
```

```
# Calculate fraction
fraction = players_with_consecutive_count / total_players if total_players > 0 else 0
```

```
# Create result DataFrame
result_df = spark.createDataFrame([
    (round(fraction, 2),)], ["fraction"])
```

```
result_df.show()
```

```
%sql
WITH first_login AS (
    SELECT
        player_id,
        MIN(event_date) as first_login_date
    FROM activity
    GROUP BY player_id),
players_with_consecutive AS (
    SELECT DISTINCT fl.player_id FROM first_login fl WHERE EXISTS (
    SELECT 1
        FROM activity a
        WHERE a.player_id = fl.player_id
        AND a.event_date = fl.first_login_date + INTERVAL '1' DAY)),
total_players AS (
    SELECT COUNT(DISTINCT player_id) as total_count FROM activity)
SELECT ROUND((SELECT COUNT(*) FROM players_with_consecutive) * 1.0 /
    NULLIF((SELECT total_count FROM total_players), 0), 2) as fraction FROM dual;
```

user_id, login_date is the primary key of this table
this table records the sessions of users who visits the badminton court
each row logs a user's activity for the day, indicating the number of sessions played
it also shows the kit_id that player has used on particular day

badminton_court - user_id, kit_id, login_date, session_count

```
%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F
```

```
# Assuming badminton_court has: user_id, kit_id, login_date, session_count
```

```
# Get user's first kit usage
window_spec = Window.partitionBy("user_id").orderBy("login_date")
```

```
first_kit_usage = df.withColumn("rn", F.row_number().over(window_spec)) \
    .filter(F.col("rn") == 1) \
    .select("user_id", "kit_id", "login_date") \
    .withColumnRenamed("kit_id", "first_kit") \
    .withColumnRenamed("login_date", "first_login_date")
```

```
# Find users who used the same kit for consecutive days
consecutive_same_kit = df.alias("a").join(
    df.alias("b"),
    (F.col("a.user_id") == F.col("b.user_id")) &
    (F.date_add(F.col("a.login_date"), 1) == F.col("b.login_date")) &
    (F.col("a.kit_id") == F.col("b.kit_id"))
```

```

).select("a.user_id", "a.kit_id", "a.login_date") \
.distinct()

# Calculate average sessions per user per kit
avg_sessions_per_kit = df.groupBy("user_id", "kit_id") \
    .agg(F.avg("session_count").alias("avg_sessions"))

# Find most used kit per user
most_used_kit = df.groupBy("user_id", "kit_id") \
    .agg(F.count("*").alias("usage_count")) \
    .withColumn("rn", F.row_number().over(
        Window.partitionBy("user_id").orderBy(F.col("usage_count").desc())
    )) \
    .filter(F.col("rn") == 1) \
    .select("user_id", "kit_id", "usage_count")

```

```

%sql
SELECT
    user_id,
    kit_id as first_kit,
    login_date as first_login_date
FROM (
    SELECT
        user_id,
        kit_id,
        login_date,
        ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY login_date
        ) as rn
    FROM badimation_court) t WHERE rn = 1;
-----

```

develop a query to identify the first badmination kit used by each player to login in.
if some player has used more than one kit on first day then show the all the kits that is used on first day.

badimation_court - user_id, kit_id, login_date, session_count
expected output - player_id, kit_id

```

%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Find first login date for each user
window_spec = Window.partitionBy("user_id").orderBy("login_date")

# Get all entries from first login date
df_with_rank = df.withColumn("rn", F.row_number().over(window_spec))

# Get the minimum login date
first_date_df = df.groupBy("user_id") \
    .agg(F.min("login_date").alias("first_login_date"))

# Get all kits used on first login date
first_kits = df.join(first_date_df,

```

```

        (df.user_id == first_date_df.user_id) &
        (df.login_date == first_date_df.first_login_date)) \
.select("user_id", "kit_id") \
.distinct() \
.orderBy("user_id", "kit_id")

```

```

%sql
WITH first_login_date AS (
    SELECT
        user_id,
        MIN(login_date) as first_login_date
    FROM badimation_court
    GROUP BY user_id
)
SELECT DISTINCT
    b.user_id,
    b.kit_id
FROM badimation_court b
JOIN first_login_date f ON b.user_id = f.user_id
    AND b.login_date = f.first_login_date
ORDER BY b.user_id, b.kit_id;

```

show the list first badmination kit used by player on their first day
badimation_court - user_id, kit_id, login_date, session_count

```

%python
first_login_details = df.join(
    df.groupBy("user_id")
        .agg(F.min("login_date").alias("first_login_date")),
    (df.user_id == user_first_login.user_id) &
    (df.login_date == user_first_login.first_login_date)
).select(
    df.user_id.alias("player_id"),
    df.kit_id,
    df.login_date.alias("first_login_date"),
    df.session_count).orderBy("player_id", "kit_id")

```

```

%sql
SELECT
    user_id as player_id,
    first_login_date,
    LISTAGG(kit_id, ', ') WITHIN GROUP (ORDER BY kit_id) as kits_used,
    COUNT(DISTINCT kit_id) as distinct_kits_count,
    SUM(session_count) as total_sessions_first_day
FROM (
    SELECT
        b.user_id,
        b.kit_id,
        b.login_date as first_login_date,
        b.session_count,
        MIN(b.login_date) OVER (PARTITION BY b.user_id) as min_login_date
    FROM badimation_court b) t
WHERE login_date = min_login_date

```

```
GROUP BY user_id, first_login_date
ORDER BY user_id;
```

write a solution to report for each player and date, how many games played so far by the player

i.e, the total number of games played by the player until that date.

badimation_court - user_id, kit_id, login_date, session_count

```
%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Window specification: partition by user, order by date
window_spec = Window.partitionBy("user_id") \
    .orderBy("login_date") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

# Calculate running total of sessions
result = df.withColumn("games_played_so_far",
    F.sum("session_count").over(window_spec)) \
    .select("user_id", "login_date", "session_count", "games_played_so_far") \
    .orderBy("user_id", "login_date")

# Show results
result.show()
```

```
%sql
SELECT
    user_id,
    login_date,
    session_count,
    SUM(session_count) OVER (
        PARTITION BY user_id
        ORDER BY login_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) as
    games_played_so_far FROM badimation_court ORDER BY user_id, login_date;
```

Users who logged in on the day immediately following their initial login date

badimation_court - user_id, kit_id, login_date, session_count

output - user_id

```
%python
first_login = df.groupBy("user_id") \
    .agg(F.min("login_date").alias("first_login_date"))

# Check if user logged in the next day
users_with_next_day = df.alias("a").join(
    first_login.alias("f"),
    F.col("a.user_id") == F.col("f.user_id")
).filter(
    F.col("a.login_date") == F.date_add(F.col("f.first_login_date"), 1)
).select("a.user_id").distinct()

# Show results
print(f"Number of users who logged in day after first login:
```



```
{users_with_next_day.count()})  
users_with_next_day.show()
```

```
%sql  
WITH first_login AS (  
    SELECT  
        user_id,  
        MIN(login_date) as first_login_date  
    FROM badimation_court  
    GROUP BY user_id)  
SELECT DISTINCT b.user_id  
FROM badimation_court b  
JOIN first_login f ON b.user_id = f.user_id  
WHERE b.login_date = f.first_login_date + INTERVAL '1' DAY  
ORDER BY b.user_id;
```

finding the top 5 movies with the highest average rating among users who signed up in the last 12 months and have watched at least 3 movies. The result should include the movie title , average rating and the number of unique users who rated the movie.

users - user_id, name, signup_date, subscription_plan
movies - movie_id, title, release_year, genre
watch_records - record_id, user_id, movie_id, watch_date, rating

```
%python  
current_date = datetime.now()  
twelve_months_ago = current_date - timedelta(days=365)  
  
result = watch_records_df.alias("w") \  
    .join(users_df.alias("u"), "user_id") \  
    .filter(F.col("u.signup_date") >= twelve_months_ago) \  
    .groupBy("w.user_id") \  
    .agg(F.countDistinct("w.movie_id").alias("movies_watched")) \  
    .filter(F.col("movies_watched") >= 3) \  
    .join(watch_records_df.alias("w2"), "user_id") \  
    .groupBy("w2.movie_id") \  
    .agg(  
        F.avg("w2.rating").alias("avg_rating"),  
        F.countDistinct("w2.user_id").alias("unique_users_count")  
    ) \  
    .join(movies_df, "movie_id") \  
    .select(  
        "title",  
        F.round("avg_rating", 2).alias("average_rating"),  
        "unique_users_count"  
    ) \  
    .orderBy(F.col("avg_rating").desc(), F.col("unique_users_count").desc()) \  
    .limit(5)
```

```
%sql  
SELECT  
    m.title,  
    ROUND(AVG(w.rating), 2) as average_rating,  
    COUNT(DISTINCT w.user_id) as unique_users_count,  
    COUNT(*) as total_ratings
```

```

FROM watch_records w
INNER JOIN movies m ON w.movie_id = m.movie_id
WHERE w.user_id IN (
    -- Users who signed up in last 12 months
    SELECT u.user_id
    FROM users u
    WHERE u.signup_date >= ADD_MONTHS(TRUNC(SYSDATE), -12)
    INTERSECT
    -- Users who watched at least 3 movies
    SELECT wr.user_id
    FROM watch_records wr
    GROUP BY wr.user_id
    HAVING COUNT(DISTINCT wr.movie_id) >= 3)
GROUP BY m.movie_id, m.title
HAVING COUNT(DISTINCT w.user_id) > 0
ORDER BY average_rating DESC, unique_users_count DESC
FETCH FIRST 5 ROWS ONLY;

```

give me an output to show records of participants who has participated more than 1 event.[event_name, participant_name]

participants - participant_id, participant_name, event_id
events - event_id, event_name

```

%python
from pyspark.sql import functions as F

# Count events per participant
participant_event_counts = participants_df.groupBy("participant_id",
"participant_name") \
    .agg(F.countDistinct("event_id").alias("event_count")) \
    .filter(F.col("event_count") > 1)

# Get event details for these participants
result = participants_df.join(
    participant_event_counts.select("participant_id"),
    "participant_id"
).join(
    events_df,
    "event_id"
).select(
    "event_name",
    "participant_name"
).distinct() \
.orderBy("participant_name", "event_name")

# Show results
print("Participants in multiple events:")
result.show(truncate=False)

%sql
SELECT
    e.event_name,
    p.participant_name

```

```

FROM participants p
JOIN events e ON p.event_id = e.event_id
WHERE p.participant_id IN (
    SELECT participant_id
    FROM participants
    GROUP BY participant_id
    HAVING COUNT(DISTINCT event_id) > 1
)
ORDER BY p.participant_name, e.event_name;

```

give a weather table, write a SQL query to find all dates'ID with higher temperature compared to its previous (yesterday's) dates

table - id, date, temperature

```

%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Create window to get previous day's temperature
window_spec = Window.orderBy("date")

# Calculate temperature difference
result = df.withColumn("prev_day_temp",
    F.lag("temperature").over(window_spec)) \
    .withColumn("temp_increase",
    F.col("temperature") - F.col("prev_day_temp")) \
    .filter(F.col("temp_increase") > 0) \
    .select("id", "date", "temperature", "prev_day_temp", "temp_increase") \
    .orderBy("date")

# Show results
print("Days with temperature higher than previous day:")
result.show()

%sql
SELECT
    id,
    date,
    temperature,
    prev_temperature,
    temperature - prev_temperature as temperature_increase
FROM (
    SELECT
        id,
        date,
        temperature,
        LAG(temperature) OVER (ORDER BY date) as prev_temperature
    FROM weather_table) t WHERE temperature > prev_temperature ORDER BY date;

```

how to transform array of array into columns? using pyspark with example

```

from pyspark.sql import SparkSession

```

```
from pyspark.sql import functions as F
from pyspark.sql.types import ArrayType, IntegerType, StringType, StructType,
StructField
```

```
spark = SparkSession.builder.appName("ArrayTransform").getOrCreate()
```

```
# Sample data: array of arrays
```

```
data = [
    (1, [[1, 2, 3], [4, 5, 6], [7, 8, 9]]),
    (2, [[10, 20, 30], [40, 50, 60]]),
    (3, [[100, 200], [300, 400], [500, 600], [700, 800]])
]
```

```
# Define schema
```

```
schema = StructType([
    StructField("id", IntegerType()),
    StructField("array_of_arrays", ArrayType(ArrayType(IntegerType())))
])
```

```
# Create DataFrame
```

```
df = spark.createDataFrame(data, schema)
df.show(truncate=False)
```

find the employee who had the longest time between their hire date and their first promotion for each department

employee - employee_id, name, department_id, salary, hire_date

department - department_id, department_name

```
%python
```

```
result_agg = employee_promotion.filter(F.col("days_to_promotion").isNotNull()) \
    .groupBy("department_id") \
    .agg(
        F.max("days_to_promotion").alias("max_days_to_promotion")
    ) \
    .join(employee_promotion, ["department_id"]) \
    .filter(F.col("days_to_promotion") == F.col("max_days_to_promotion")) \
    .join(department_df, "department_id") \
    .select(
        "department_name",
        "employee_id",
        "name",
        "hire_date",
        "first_promotion_date",
        "days_to_promotion"
    ) \
    .orderBy("department_name")
```

```
%sql
```

```
WITH emp_promo AS (
    SELECT
        e.employee_id,
        e.name,
```

```

        e.department_id,
        e.hire_date,
        (SELECT MIN(promotion_date)
         FROM promotions p
         WHERE p.employee_id = e.employee_id) as first_promotion_date
    FROM employee e
)
SELECT
    d.department_name,
    ep.employee_id,
    ep.name,
    ep.hire_date,
    ep.first_promotion_date,
    ep.first_promotion_date - ep.hire_date as days_to_promotion
FROM emp_promo ep
JOIN department d ON ep.department_id = d.department_id
WHERE ep.first_promotion_date IS NOT NULL
AND (ep.department_id, ep.first_promotion_date - ep.hire_date) IN (
SELECT department_id, MAX(first_promotion_date - hire_date) FROM emp_promo
WHERE first_promotion_date IS NOT NULL GROUP BY department_id) ORDER BY
d.department_name;

```

find the employee who got promoted and then immediately took a vacation

employee- employee_id, name, department_id, salary, hire_date
 promotions - promotion_id, employee_id, promotion_date, new_salary, new_title
 leaves - leave_id, employee_id, leave_start, leave_end

```

%python
promotion_vacation = promotions_df.alias("p").join(
    leaves_df.alias("l"),
    (F.col("p.employee_id") == F.col("l.employee_id")) &
    (F.col("l.leave_start") >= F.col("p.promotion_date"))).withColumn(
    "days_after_promotion",
    F.datediff(F.col("leave_start"), F.col("promotion_date"))).filter(
    F.col("days_after_promotion") <= 7).select(
    F.col("p.employee_id"),
    F.col("p.promotion_id"),
    F.col("p.promotion_date"),
    F.col("p.new_title"),
    F.col("l.leave_id"),
    F.col("l.leave_start"),
    F.col("l.leave_end"),
    F.col("days_after_promotion"))

```

```

# Join with employee details
result = promotion_vacation.join(
    employee_df,
    "employee_id").select(
    "employee_id",
    "name",
    "department_id",
    "promotion_date",
    "new_title",
    "leave_start",

```

```

    "leave_end",
    "days_after_promotion").orderBy("promotion_date")

# Show results
print("Employees who took vacation soon after promotion:")
result.show()

%sql
WITH promotion_vacation AS (
    SELECT
        p.employee_id,
        p.promotion_id,
        p.promotion_date,
        p.new_title,
        l.leave_id,
        l.leave_start,
        l.leave_end,
        l.leave_start - p.promotion_date as days_after_promotion
    FROM promotions p
    JOIN leaves l ON p.employee_id = l.employee_id
    WHERE l.leave_start >= p.promotion_date
    AND l.leave_start - p.promotion_date <= 7 -- Within 7 days
)
SELECT
    e.employee_id,
    e.name,
    e.department_id,
    pv.promotion_date,
    pv.new_title,
    pv.leave_start,
    pv.leave_end,
    pv.days_after_promotion
FROM promotion_vacation pv
JOIN employee e ON pv.employee_id = e.employee_id
ORDER BY pv.promotion_date;

```

write an SQL query to find all weight that appear at least three times consecutively
columns - id, weight

```

%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Method 1: Using window functions with LAG and LEAD
window_spec = Window.orderBy("id")

df_with_neighbors = df.withColumn("prev_weight1", F.lag("weight",
1).over(window_spec)) \
    .withColumn("prev_weight2", F.lag("weight", 2).over(window_spec)) \
    .withColumn("next_weight1", F.lead("weight", 1).over(window_spec)) \
    .withColumn("next_weight2", F.lead("weight", 2).over(window_spec))

# Check for sequences of 3 consecutive same weights
result = df_with_neighbors.filter(

```

```

((F.col("weight") == F.col("prev_weight1")) &
 (F.col("weight") == F.col("prev_weight2"))) |
((F.col("weight") == F.col("prev_weight1")) &
 (F.col("weight") == F.col("next_weight1"))) |
((F.col("weight") == F.col("next_weight1")) &
 (F.col("weight") == F.col("next_weight2")))
).select("id", "weight").orderBy("id")

# Alternative: More robust method using grouping
window_spec_grp = Window.orderBy("id")

# Create groups of consecutive same values
df_with_groups = df.withColumn("is_same_as_prev",
                               F.when(F.col("weight") == F.lag("weight", 1).over(window_spec_grp),
                                0).otherwise(1)) \
    .withColumn("group_id", F.sum("is_same_as_prev").over(window_spec_grp))

# Count consecutive occurrences
consecutive_counts = df_with_groups.groupBy("group_id", "weight") \
    .agg(F.count("*").alias("consecutive_count")) \
    .filter(F.col("consecutive_count") >= 3)

# Get original rows that are part of sequences
result_robust = df_with_groups.join(
    consecutive_counts.select("group_id",
    "group_id"
).select("id", "weight").orderBy("id")

# Show results
print("Weights appearing at least 3 times consecutively:")
result_robust.show()

```

```

%sql
SELECT DISTINCT id, weight
FROM (
    SELECT
        id,
        weight,
        LAG(weight, 1) OVER (ORDER BY id) as prev1,
        LAG(weight, 2) OVER (ORDER BY id) as prev2,
        LEAD(weight, 1) OVER (ORDER BY id) as next1,
        LEAD(weight, 2) OVER (ORDER BY id) as next2
    FROM weights_table
) t
WHERE (weight = prev1 AND weight = prev2) -- Current is third in sequence
    OR (weight = prev1 AND weight = next1) -- Current is middle of sequence
    OR (weight = next1 AND weight = next2) -- Current is first in sequence
ORDER BY id;

```

write an SQL query for a report that provides the pairs (actor_id, director_id) where the actor has cooperated with the director at least three times

columns - actor_id, director_id, timestamp

```

%pyspark
from pyspark.sql import functions as F

```

```
# Essential query
result = df.groupBy("actor_id", "director_id") \
    .agg(F.count("*").alias("cooperation_count")) \
    .filter(F.col("cooperation_count") >= 3) \
    .select("actor_id", "director_id") \
    .orderBy("actor_id", "director_id")
```

```
result.show()
```

```
%sql
-- Essential query
SELECT
    actor_id,
    director_id,
    COUNT(*) as cooperation_count
FROM collaborations
GROUP BY actor_id, director_id
HAVING COUNT(*) >= 3
ORDER BY actor_id, director_id;
```

```
----
write pyspark code
transform the productID col if after 3rd character any word occur
create new column available flag
when inventoryflag is yes then availableFlag is 1
when inventoryflag is no then check for previous weekOfYear forsame product, if yes
there then availableFlag is 1 else 0
```

```
columns - custID, product_id, inventoryFlag, weekOfYear, yearMonth
```

```
%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F
from pyspark.sql.types import IntegerType
```

```
# Sample data creation (if needed)
data = [
    ("C001", "ABC123XYZ", "yes", 10, "2024-03"),
    ("C001", "ABC123XYZ", "no", 11, "2024-03"),
    ("C001", "ABC123XYZ", "yes", 12, "2024-03"),
    ("C002", "DEF456", "no", 10, "2024-03"),
    ("C002", "DEF456", "no", 11, "2024-03"),
    ("C002", "DEF456", "yes", 12, "2024-03"),
    ("C003", "GHI789ABC", "no", 10, "2024-03"),
    ("C003", "GHI789ABC", "no", 11, "2024-03"),
    ("C003", "GHI789ABC", "no", 12, "2024-03"),
    ("C004", "JKL012", "no", 10, "2024-03"),
    ("C004", "JKL012", "no", 11, "2024-03")
]
```

```
schema = ["custID", "product_id", "inventoryFlag", "weekOfYear", "yearMonth"]
df = spark.createDataFrame(data, schema)
```

```
# Step 1: Transform productID - Check if any word occurs after 3rd character
# Assuming "word" means alphabetic characters (not digits)
```



```

df_transformed = df.withColumn(
    "hasWordAfterThirdChar",
    F.when(
        F.length("product_id") > 3,
        F.regexp_extract(F.col("product_id").substr(4, 100), "[A-Za-z]+", 0) != ""
    ).otherwise(F.lit(False))
).withColumn(
    "transformed_product_id",
    F.when(F.col("hasWordAfterThirdChar"), F.concat(F.lit("MOD_"), F.col("product_id")))
    .otherwise(F.col("product_id"))
)

```

```

# Step 2: Window specification for previous week lookup
window_spec = Window.partitionBy("custID", "transformed_product_id") \
    .orderBy("yearMonth", "weekOfYear")

```

```

# Step 3: Get previous week's inventory flag
df_with_prev = df_transformed.withColumn(
    "prev_week_flag",
    F.lag("inventoryFlag", 1).over(window_spec)
).withColumn(
    "prev_week",
    F.lag("weekOfYear", 1).over(window_spec)
)

```

```

# Step 4: Create availableFlag based on business rules
df_result = df_with_prev.withColumn(
    "availableFlag",
    F.when(F.col("inventoryFlag") == "yes", 1)
    .when(
        (F.col("inventoryFlag") == "no") &
        (F.col("prev_week_flag") == "yes") &
        (F.col("prev_week") == F.col("weekOfYear") - 1),
        1
    )
    .otherwise(0)
).select(
    "custID",
    "product_id",
    "transformed_product_id",
    "inventoryFlag",
    "weekOfYear",
    "yearMonth",
    "availableFlag"
)

```

```

# Show results
print("Transformed Data with Availability Flag:")
df_result.orderBy("custID", "transformed_product_id", "weekOfYear").show()

```

find the number of session completed by users that had their very first session as viewers. You should just return their user ID and number of sessions.
columns - user_id, session_start, session_end, session_id, session_type

```
%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Find first session type for each user
window_spec = Window.partitionBy("user_id").orderBy("session_start")

first_session_users = df.withColumn("rn", F.row_number().over(window_spec)) \
    .filter(F.col("rn") == 1) \
    .filter(F.col("session_type") == "viewer") \
    .select("user_id") \
    .distinct()

# Count total sessions for these users
result = df.join(first_session_users, "user_id") \
    .groupBy("user_id") \
    .agg(F.count("*").alias("total_sessions")) \
    .orderBy("user_id")

# Show results
result.show()
```

```
%sql
WITH first_session AS (
    SELECT
        user_id,
        session_type,
        ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY session_start
        ) as rn
    FROM sessions
)
SELECT
    s.user_id,
    COUNT(*) as total_sessions
FROM sessions s
WHERE s.user_id IN (
    SELECT user_id
    FROM first_session
    WHERE rn = 1
    AND session_type = 'viewer'
)
GROUP BY s.user_id
ORDER BY s.user_id;
```

write a query to retrieve the count of companies that have posted duplicate job listings

job_listing - job_id, company_id, title, description

```
%python
duplicate_companies = df.groupBy("company_id", "title", "description") \
    .agg(F.count("*").alias("listing_count")) \
```

```
.filter(F.col("listing_count") > 1) \
.select("company_id") \
.distinct()
```

%sql

```
SELECT COUNT(DISTINCT company_id) as companies_with_duplicates
FROM (
    SELECT company_id, title, description
    FROM job_listing
    GROUP BY company_id, title, description
    HAVING COUNT(*) > 1);
```

an active user is defined as a user who has performed actions such as 'sign_in', 'like' or 'comment' in both the current_month and previous_month

user_actions - user_id, event_id, event_type, event_date

%python

```
from pyspark.sql import functions as F
from datetime import datetime, timedelta
```

Get current date and calculate previous month

```
current_date = datetime.now()
current_month = current_date.strftime('%Y-%m')
prev_month = (current_date.replace(day=1) - timedelta(days=1)).strftime('%Y-%m')
```

Extract month from event_date

```
df_with_month = df.withColumn("event_month", F.date_format("event_date",
"yyyy-MM"))
```

Filter for relevant events and months

```
relevant_events = df_with_month.filter(
    (F.col("event_type").isin(['sign_in', 'like', 'comment'])) &
    (F.col("event_month").isin([current_month, prev_month]))
)
```

Find users active in both months

```
active_users = relevant_events.groupBy("user_id") \
    .agg(F.collect_set("event_month").alias("active_months")) \
    .filter(F.size(F.col("active_months")) == 2) \
    .select("user_id")
```

Show results

```
print(f'Active users in both {prev_month} and {current_month}: {active_users.count()}')
active_users.show()
```

%sql

```
SELECT user_id
FROM (
    SELECT DISTINCT user_id
    FROM user_actions
    WHERE event_type IN ('sign_in', 'like', 'comment')
    AND TO_CHAR(event_date, 'YYYY-MM') = TO_CHAR(ADD_MONTHS(TRUNC(SYSDATE,
'MM'), -1), 'YYYY-MM')
```

```

)
INTERSECT
SELECT user_id
FROM (
    SELECT DISTINCT user_id
    FROM user_actions
    WHERE event_type IN ('sign_in', 'like', 'comment')
    AND TO_CHAR(event_date, 'YYYY-MM') = TO_CHAR(TRUNC(SYSDATE, 'MM'),
'YYYY-MM')
);

```

customers - customer_name, customer_email, phone_number, address
orders - customer_id, order_date, total_amount, payment_method, delivery_address,
city, state, delievery_date, delivery_state
order_items - order_id, product_id, quality, unit_price, total_price
products - product_name, category, brand, price, description

1) which product category generate the highest revenue for the last month

```

%python
from pyspark.sql import functions as F
from datetime import datetime, timedelta

# Calculate last month
current_date = datetime.now()
last_month_start = (current_date.replace(day=1) - timedelta(days=1)).replace(day=1)
last_month_end = current_date.replace(day=1) - timedelta(days=1)
last_month_str = last_month_start.strftime('%Y-%m')

# Join all necessary tables
revenue_by_category = orders_df.join(
    order_items_df, "order_id"
).join(
    products_df, "product_id"
).filter(
    (F.col("order_date") >= last_month_start) &
    (F.col("order_date") <= last_month_end)
).groupBy("category") \
.agg(
    F.sum("total_price").alias("total_revenue"),
    F.countDistinct("order_id").alias("order_count"),
    F.sum("quality").alias("total_quantity")
).orderBy(F.col("total_revenue").desc())

# Get top category
top_category = revenue_by_category.first()
print(f"Top category for {last_month_str}: {top_category['category']}")
print(f"Revenue: ${top_category['total_revenue']:,.2f}")

revenue_by_category.show()

%sql
-- Top category by revenue for last month
WITH last_month_data AS (
    SELECT

```

```

        TRUNC(SYSDATE, 'MONTH') - INTERVAL '1' MONTH as month_start,
        LAST_DAY(TRUNC(SYSDATE, 'MONTH') - INTERVAL '1' MONTH) as month_end
    FROM dual
),
category_revenue AS (
    SELECT
        p.category,
        SUM(oi.total_price) as total_revenue,
        COUNT(DISTINCT o.order_id) as order_count,
        SUM(oi.quantity) as total_quantity,
        ROW_NUMBER() OVER (ORDER BY SUM(oi.total_price) DESC) as rank
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    CROSS JOIN last_month_data lmd
    WHERE o.order_date BETWEEN lmd.month_start AND lmd.month_end
    GROUP BY p.category
)
SELECT
    category,
    total_revenue,
    order_count,
    total_quantity
FROM category_revenue
WHERE rank = 1;

```

2) List the top 3 brand among the top 1 category generating the highest

```

%python
# Get top category from previous query
top_category_name = top_category['category']

# Find top 3 brands in top category
top_brands = orders_df.join(
    order_items_df, "order_id"
).join(
    products_df, "product_id"
).filter(
    (F.col("order_date") >= last_month_start) &
    (F.col("order_date") <= last_month_end) &
    (F.col("category") == top_category_name)
).groupBy("brand") \
    .agg(
        F.sum("total_price").alias("brand_revenue"),
        F.countDistinct("order_id").alias("order_count"),
        F.sum("quantity").alias("total_quantity"),
        F.countDistinct("product_id").alias("unique_products")
    ).orderBy(F.col("brand_revenue").desc()) \
    .limit(3)

print(f"Top 3 brands in {top_category_name}:")
top_brands.show()

```

```

%sql
-- Top 3 brands in the top category

```

```

WITH last_month_data AS (
    SELECT
        TRUNC(SYSDATE, 'MONTH') - INTERVAL '1' MONTH as month_start,
        LAST_DAY(TRUNC(SYSDATE, 'MONTH') - INTERVAL '1' MONTH) as month_end
    FROM dual
),
top_category AS (
    SELECT
        p.category,
        SUM(oi.total_price) as total_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    CROSS JOIN last_month_data lmd
    WHERE o.order_date BETWEEN lmd.month_start AND lmd.month_end
    GROUP BY p.category
    ORDER BY total_revenue DESC
    FETCH FIRST 1 ROW ONLY
),
top_brands AS (
    SELECT
        p.brand,
        SUM(oi.total_price) as brand_revenue,
        COUNT(DISTINCT o.order_id) as order_count,
        SUM(oi.quantity) as total_quantity,
        COUNT(DISTINCT p.product_id) as unique_products,
        ROW_NUMBER() OVER (ORDER BY SUM(oi.total_price) DESC) as brand_rank
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    CROSS JOIN last_month_data lmd
    CROSS JOIN top_category tc
    WHERE o.order_date BETWEEN lmd.month_start AND lmd.month_end
    AND p.category = tc.category
    GROUP BY p.brand
)
SELECT
    brand,
    brand_revenue,
    order_count,
    total_quantity,
    unique_products
FROM top_brands
WHERE brand_rank <= 3
ORDER BY brand_rank;

```

3) what is the average order value of customers who make at least 10 transactions in the last month

```

%python
# Find customers with 10+ transactions in last month
frequent_customers = orders_df.filter(
    (F.col("order_date") >= last_month_start) &
    (F.col("order_date") <= last_month_end)
).groupBy("customer_id") \

```

```

.agg(F.count("*").alias("transaction_count")) \
.filter(F.col("transaction_count") >= 10) \
.select("customer_id")

# Calculate AOV for these customers
aov_frequent_customers = orders_df.join(
    frequent_customers, "customer_id"
).filter(
    (F.col("order_date") >= last_month_start) &
    (F.col("order_date") <= last_month_end)
).groupBy("customer_id") \
.agg(
    F.avg("total_amount").alias("average_order_value"),
    F.count("*").alias("transaction_count"),
    F.sum("total_amount").alias("total_spent")
).orderBy(F.col("average_order_value").desc())

# Overall AOV for frequent customers
overall_aov = aov_frequent_customers.agg(
    F.avg("average_order_value").alias("overall_avg_order_value"),
    F.count("*").alias("frequent_customer_count"),
    F.sum("total_spent").alias("total_revenue_from_frequent")
).first()

print(f"Overall AOV for customers with 10+ transactions:
${overall_aov['overall_avg_order_value']:.2f}")
print(f"Number of frequent customers: {overall_aov['frequent_customer_count']}")

aov_frequent_customers.show(10)

%sql
-- AOV for customers with 10+ transactions in last month
WITH last_month_data AS (
    SELECT
        TRUNC(SYSDATE, 'MONTH') - INTERVAL '1' MONTH as month_start,
        LAST_DAY(TRUNC(SYSDATE, 'MONTH') - INTERVAL '1' MONTH) as month_end
    FROM dual
),
frequent_customers AS (
    SELECT
        customer_id,
        COUNT(*) as transaction_count
    FROM orders
    CROSS JOIN last_month_data lmd
    WHERE order_date BETWEEN lmd.month_start AND lmd.month_end
    GROUP BY customer_id
    HAVING COUNT(*) >= 10
),
customer_aov AS (
    SELECT
        o.customer_id,
        AVG(o.total_amount) as average_order_value,
        COUNT(*) as transaction_count,
        SUM(o.total_amount) as total_spent
    FROM orders o

```

```

JOIN frequent_customers fc ON o.customer_id = fc.customer_id
CROSS JOIN last_month_data lmd
WHERE o.order_date BETWEEN lmd.month_start AND lmd.month_end
GROUP BY o.customer_id
)
SELECT
    ROUND(AVG(average_order_value), 2) as overall_avg_order_value,
    COUNT(*) as frequent_customer_count,
    SUM(total_spent) as total_revenue_from_frequent,
    MIN(average_order_value) as min_aov,
    MAX(average_order_value) as max_aov,
    ROUND(STDDEV(average_order_value), 2) as aov_std_dev
FROM customer_aov;

```

the company wants to identify customers who have placed orders for the same product multiple times within the past six months. This information can be used for targeted marketing campaign to promote related products or offer discounts to loyal customers.

customer - customer_id, first_name, last_name, email
 product_id, product_name, product_category, unit_price
 orders - order_id, customer_id, order_date, product_id, quantity_ordered

```

%python
from pyspark.sql import functions as F
from datetime import datetime, timedelta

# Calculate date 6 months ago
six_months_ago = datetime.now() - timedelta(days=180)

# Join orders with customers and products
orders_with_details = orders_df.join(
    customers_df, "customer_id").join(
    products_df, "product_id").filter(
    F.col("order_date") >= six_months_ago)

# Find customers who ordered same product multiple times
loyal_customers = orders_with_details.groupBy(
    "customer_id", "first_name", "last_name", "email", "product_id", "product_name").agg(
    F.count("*").alias("order_count"),
    F.sum("quantity_ordered").alias("total_quantity"),
    F.max("order_date").alias("last_order_date"),
    F.min("order_date").alias("first_order_date")).filter(
    F.col("order_count") >= 2).withColumn(
    "days_between_orders",
    F.datediff(F.col("last_order_date"), F.col("first_order_date"))).orderBy(
    F.col("order_count").desc(),
    F.col("total_quantity").desc())

print(f"Found {loyal_customers.count()} customer-product relationships with repeat orders")
loyal_customers.show()

```



```

%sql
-- Customers who ordered same product multiple times in last 6 months
WITH six_month_orders AS (
    SELECT
        o.customer_id,
        o.product_id,
        COUNT(*) as order_count,
        SUM(o.quantity_ordered) as total_quantity,
        MAX(o.order_date) as last_order_date,
        MIN(o.order_date) as first_order_date
    FROM orders o
    WHERE o.order_date >= ADD_MONTHS(SYSDATE, -6)
    GROUP BY o.customer_id, o.product_id
    HAVING COUNT(*) >= 2
)
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    c.email,
    p.product_id,
    p.product_name,
    p.product_category,
    smo.order_count,
    smo.total_quantity,
    smo.first_order_date,
    smo.last_order_date,
    smo.last_order_date - smo.first_order_date as days_between_orders
FROM six_month_orders smo
JOIN customers c ON smo.customer_id = c.customer_id
JOIN products p ON smo.product_id = p.product_id
ORDER BY smo.order_count DESC, smo.total_quantity DESC;

```

find teh employee with second highest salary
emp - dept, emp_name, salary

```

%python
window_spec = Window.orderBy(F.col("salary").desc())

result = df.withColumn("rank", F.dense_rank().over(window_spec)) \
    .filter(F.col("rank") == 2) \
    .select("dept", "emp_name", "salary")

```

```

%sql
SELECT dept, emp_name, salary
FROM (
    SELECT dept, emp_name, salary,
        ROW_NUMBER() OVER (ORDER BY salary DESC) as rn
    FROM emp) WHERE rn = 2;

```

write a query that gets the number of retained users per month, in the case retention for a given month is defined as the number of users who logged in the month who also logged in the immediately previous month.
table - user_id, date

```

%python
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Add month column
df_with_month = df.withColumn("month", F.date_format("date", "yyyy-MM"))

# Get distinct user-month combinations
user_months = df_with_month.select("user_id", "month").distinct()

# Create window to get previous month
window_spec = Window.partitionBy("user_id").orderBy("month")

# Add previous month
user_with_prev = user_months.withColumn("prev_month",
                                         F.lag("month", 1).over(window_spec))

# Calculate retention
retention = user_with_prev.filter(
    (F.col("prev_month").isNotNull()) &
    (F.months_between(F.to_date(F.concat(F.col("month"), "-01")),
                      F.to_date(F.concat(F.col("prev_month"), "-01"))) == 1)
).groupBy("month") \
  .agg(F.countDistinct("user_id").alias("retained_users")) \
  .orderBy("month")

# Add total active users per month for context
active_users = user_months.groupBy("month") \
  .agg(F.countDistinct("user_id").alias("active_users"))

result = active_users.join(retention, "month", "left") \
  .fillna(0, subset=["retained_users"]) \
  .withColumn("retention_rate",
             F.round(F.col("retained_users") / F.col("active_users") * 100, 2)) \
  .orderBy("month")

%sql
SELECT
    TO_CHAR(curr.month_date, 'YYYY-MM') as month,
    COUNT(DISTINCT curr.user_id) as retained_users
FROM (
    SELECT DISTINCT
        user_id,
        TRUNC(date, 'MM') as month_date
    FROM user_logins
) curr
JOIN (
    SELECT DISTINCT
        user_id,
        TRUNC(date, 'MM') as month_date
    FROM user_logins
) prev ON curr.user_id = prev.user_id
      AND curr.month_date = ADD_MONTHS(prev.month_date, 1)
GROUP BY TO_CHAR(curr.month_date, 'YYYY-MM')

```

ORDER BY month;

write a query to get the pair of state with total streaming amount within 1000 of each other. for the snippet above, we would want to see something like:
column - state, streams, month

%python

```
result = comparison.select(
    "month",
    F.concat(F.col("state1"), F.lit("-"), F.col("state2")).alias("state_pair"),
    F.col("streams1"),
    F.col("streams2"),
    "stream_diff").orderBy("month", "stream_diff")
```

%sql

```
SELECT
    month,
    state1 || '-' || state2 as state_pair,
    streams1,
    streams2,
    stream_diff
FROM (
    SELECT
        a.month,
        a.state as state1,
        a.total_streams as streams1,
        b.state as state2,
        b.total_streams as streams2,
        ABS(a.total_streams - b.total_streams) as stream_diff,
        ROW_NUMBER() OVER (
            PARTITION BY a.month, a.state
            ORDER BY ABS(a.total_streams - b.total_streams)
        ) as rn
    FROM state_monthly_totals a
    JOIN state_monthly_totals b ON a.month = b.month
    AND a.state != b.state
    WHERE ABS(a.total_streams - b.total_streams) <= 1000)
WHERE rn = 1 -- Get closest match for each state
ORDER BY month, stream_diff;
```

write a query to find users who transactions has breached their credit limit

user - user_id, user_name, credit_limit

transaction - trans_id, paid_by, paid_to, amount , trans_date

%python

```
from pyspark.sql.window import Window
import pyspark.sql.functions as F
```

window_spec =

```
Window.partitionBy("paid_by").orderBy("trans_date").rowBetween(Window.unbounded
Preceding, Window.currentRow)
```

```
result = transaction_df.join(user_df, transaction_df.paid_by == user_df.user_id, "inner")
.withColumn("running_total",
```

```
F.sum("amount").over(window_spec)).filter(col("running_total") >
col("credit_limit")).select("user_id", "user_name", "credit_limit", "running_total",
"trans_date").distinct()
```

```
%sql
```

```
WITH user_transactions AS (
  SELECT
    u.user_id,
    u.user_name,
    u.credit_limit,
    t.trans_date,
    t.amount,
    SUM(t.amount) OVER (
      PARTITION BY t.paid_by
      ORDER BY t.trans_date
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) as running_total
  FROM users u JOIN transactions t ON u.user_id = t.paid_by)
SELECT DISTINCT
  user_id,
  user_name,
  credit_limit,
  MAX(running_total) as total_spent,
  MAX(trans_date) as last_breach_date FROM user_transactions WHERE running_total
> credit_limit
GROUP BY user_id, user_name, credit_limit;
```

```
----
```

write an SQL query to report the customer ids from the customer table that bought all the products in the product table.

customer - customer_id, product_key

product - product_key

```
%sql
```

```
SELECT customer_id
FROM customer_table
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(*) FROM product_table);
```

```
%python
```

```
# Count total products
```

```
total_products = product_df.count()
```

```
# Count distinct products per customer
```

```
customer_product_counts = customer_df.groupby("customer_id") \
    .agg(F.countDistinct("product_key").alias("bought_products"))
```

```
# Find customers who bought all products
```

```
result = customer_product_counts.filter(col("bought_products") == total_products) \
    .select("customer_id")
```

```
---
```

write an SQL query that reports the buyers who have bought S8 but not iphone. Note that S8 and iphone are products present in the product

product_table - product_id, product_name, unit_price

sales_table - seller_id, product_id, buyer_id, sale_date, quantity, price

```
%sql
SELECT s.buyer_id
FROM sales_table s
JOIN product_table p ON s.product_id = p.product_id GROUP BY s.buyer_id HAVING
    MAX(CASE WHEN p.product_name = 'S8' THEN 1 ELSE 0 END) = 1
    AND MAX(CASE WHEN p.product_name = 'iPhone' THEN 1 ELSE 0 END) = 0;
```

```
%python
# Get buyers who bought S8
s8_buyers = sales_df.join(product_df, "product_id") \
    .filter(col("product_name") == "S8") \
    .select("buyer_id").distinct()

# Get buyers who bought iPhone
iphone_buyers = sales_df.join(product_df, "product_id") \
    .filter(col("product_name") == "iPhone") \
    .select("buyer_id").distinct()

# Find buyers in S8 but not in iPhone
result = s8_buyers.exceptAll(iphone_buyers)

# Alternative: Single query approach
result = sales_df.join(product_df, "product_id") \
    .groupBy("buyer_id") \
    .agg(
        F.max(F.when(col("product_name") == "S8", 1).otherwise(0)).alias("bought_s8"),
        F.max(F.when(col("product_name") == "iPhone",
1).otherwise(0)).alias("bought_iphone")
    ) \
    .filter((col("bought_s8") == 1) & (col("bought_iphone") == 0)) \
    .select("buyer_id")
```